RockVsMine: Sonar Rock vs Mine Classifier

Project documentation. Team: Shivam Kumar (Backend), Rahul Singh (Frontend)

1. Introduction

RockVsMine is a machine learning project that classifies a sonar echo as either a **Rock (R)** or a **Mine (M)**. A sonar sends sound waves underwater and records the echo. A metal cylinder and a similarly shaped rock return echoes that differ only subtly, and the signal is noisy and high-dimensional, so telling them apart by hand needs trained experts and is slow. This project automates the decision with a trained model.

2. Objective

- Load and explore the UCI Sonar dataset using Python.
- Train a classification model that separates Rock from Mine.
- Evaluate the model on data it has never seen.
- Use the trained model to predict the class of a brand-new sonar reading.

Why it matters: automatic detection of underwater mines can make naval navigation and demining safer. A missed mine is more dangerous than a false alarm, which is why the future work section proposes tracking recall on the Mine class.

3. Dataset

Property	Value
Source	UCI Machine Learning Repository, Connectionist Bench (Sonar, Mines vs. Rocks)
Samples	208 (111 Mine, 97 Rock)
Features	60 numeric values per sample
Feature meaning	Energy of the echo within a frequency band, scaled from 0.0 to 1.0
Label	Last column: `R` = Rock, `M` = Mine

Each row is one sonar reading. The file has no header row, so it is loaded with `header=None`, giving a table of shape (208, 61).

4. Technology Stack

Tool	Purpose
Python 3.8+	Programming language
NumPy	Creating and reshaping arrays, including the input signal used for prediction
Pandas	Loading the CSV and working with DataFrames
scikit-learn	`train_test_split` to split data, `LogisticRegression` as the model, `accuracy_score` to measure performance

5. Methodology

The project follows a standard supervised learning pipeline:

1. **Load data:** read `sonar_data.csv` into a Pandas DataFrame.
2. **Separate features and label:** columns 0 to 59 become `X`, column 60 becomes `Y`.
3. **Split the data:** 90% for training and 10% for testing, using `stratify=Y` so Rock and Mine appear in the same ratio in both sets, and `random_state=1` so results can be repeated.
4. **Train:** fit a Logistic Regression model on the training set.
5. **Evaluate:** compute accuracy on the training set and on the unseen test set.
6. **Predict:** convert a new 60-value reading to a NumPy array, reshape it to one sample with `reshape(1, -1)`, and ask the model for the class.

6. Implementation Summary

Step	Code
Load	`pd.read_csv('sonar_data.csv', header=None)`

Step	Code
Features	`X = sonar_data.drop(columns=60, axis=1)`
Label	`Y = sonar_data[60]`
Split	`train_test_split(X, Y, test_size=0.1, stratify=Y, random_state=1)`
Train	`model = LogisticRegression(); model.fit(X_train, Y_train)`
Evaluate	`accuracy_score(model.predict(X_test), Y_test)`
Predict	`model.predict(np.asarray(input_data).reshape(1, -1))`

The prediction step prints "The object is a ROCK" when the model returns `R` and "The object is a MINE" otherwise.

7. Results

Dataset	Accuracy
Training data	83.4%
Test data	76.1%

Reading the numbers. The test set holds about 21 samples, so 76.1% corresponds to roughly 16 correct predictions out of 21. With so few test samples, one extra mistake moves the score by almost 5 points, so this figure is a rough estimate and not a precise measure.

The model scores about 7 points lower on unseen data than on training data. This is a mild gap and is normal for a small dataset with 60 features.

8. Limitations

- **Small dataset:** 208 samples limits how well any model can generalise.

- **Single split:** one train/test split makes the result depend on which samples landed in the test set.
- **Accuracy only:** accuracy hides which class the errors come from. It does not show how many mines were missed.
- **One model:** only Logistic Regression has been tried, so there is no comparison against other algorithms.
- **No feature scaling or tuning:** default settings were used throughout.

9. Future Work

- Compare more models: SVM, Random Forest, KNN.
- Add a confusion matrix, precision and recall, with special attention to recall on the Mine class.
- Use cross-validation and hyperparameter tuning for more reliable scores.
- Build a web interface where a user enters readings and receives a prediction with a confidence score.
- Visualise Rock and Mine signals side by side.
- Save the trained model so it does not need retraining on every run.

10. How to Run

```
git clone https://github.com/<your-username>/RockVsMine.git
cd RockVsMine
pip install numpy pandas scikit-learn
python rock_vs_mine.py
```

Project structure

```
RockVsMine/
├── sonar_data.csv      # Dataset
├── rock_vs_mine.py     # Main ML script
├── requirements.txt    # Dependencies
└── README.md           # Project overview
```

11. Team

Name	Role	GitHub
Shivam Kumar	Backend	@officegitshivam
Rahul Singh	Frontend	@rahulsingh2008